# PROMPT THỰC CHIẾN HẰNG NGÀY: PHƯƠNG PHÁP HIỆN ĐẠI & DỄ DÙNG NHẤT

Dành cho AI Engineers & Lập trình viên: Tối ưu hóa hiệu suất làm việc 10x với AI trong mọi tác vụ thường nhật.

## 1. CÔNG THỨC VÀNG DỄ NHỚ NHẤT: T - C - R - E (DÙNG CHO 90% CÔNG VIỆC)

| Thành phần | Ý nghĩa & Câu thần chú | Ví dụ ngắn gọn |
|---|---|---|
| **T - Task (Việc cần làm)** | Nói rõ chính xác hành động muốn AI làm (bắt đầu bằng động từ). | 'Viết hàm FastAPI upload file và stream data về S3...' |
| **C - Context (Bối cảnh)** | Stack công nghệ, version, data format, đầu vào có gì. | 'Dùng Python 3.11, boto3, nhận file PDF/DOCX tối đa 50MB.' |
| **R - Rules (Ràng buộc)** | Những điều cấm hoặc bắt buộc tuân theo. | 'Bắt buộc try-catch đầy đủ, type hinting, không dùng thư viện ngoài.' |
| **E - Example (Mẫu mong muốn)** | Output mẫu (JSON schema, format bảng, mẫu response). | 'Format JSON trả về: {"status": "success", "file_url": "..."}' |

## 2. TOP 5 PROMPT MẪU 'COPY & DÙNG NGAY' CHO CÔNG VIỆC HẰNG NGÀY

| Tình huống hằng ngày | Prompt mẫu chuẩn (Copy & Paste vào AI) |
|---|---|
| 1. Debug lỗi / Bug fix<br>Khi gặp lỗi không hiểu nguyên nhân | 'Tôi đang gặp lỗi sau khi chạy hàm [Tên hàm]:<br>• Lỗi/Log: [Dán Full Stacktrace]<br>• Input đầu vào: [Dữ liệu test]<br>-> Hãy: 1. Giải thích ngắn gọn nguyên nhân gốc rễ (Root cause). 2. Đưa code sửa lỗi giữ nguyên signature. 3. Thêm test case để không bị lặp lại.' |
| 2. Refactor & Tối ưu code<br>Code chạy chậm / khó bảo trì | 'Hãy đóng vai trò Tech Lead. Đánh giá đoạn code sau theo tiêu chuẩn SOLID & Clean Code:<br>[Dán code]<br>-> Chỉ ra 3 điểm nghẽn lớn nhất (Performance, Memory, Readability) và viết lại bản tối ưu hơn có benchmark dự kiến.' |
| 3. Thiết kế tính năng mới<br>Lên kế hoạch trước khi code | 'Tôi muốn làm tính năng [Mô tả tính năng] cho hệ thống [Công nghệ: Node/Python...].<br>-> Đừng viết code vội. Hãy hỏi tôi 3–5 câu hỏi làm rõ (Edge cases, volume, security) và đề xuất 2 phương án kiến trúc (Kèm Pro/Con).' |
| 4. Viết Unit Test & Edge Cases<br>Tăng test coverage | 'Viết bộ Unit Test bằng pytest cho hàm dưới đây:<br>[Dán code]<br>-> Yêu cầu bao phủ: Case chuẩn (Happy path), Case biên (Edge cases, None, Empty, Out-of-bound) và Mock external calls.' |
| 5. Đọc hiểu & Tóm tắt Doc/Code<br>Khi nhận codebase lạ | 'Giải thích luồng hoạt động của module này cho người mới tiếp cận:<br>[Dán code/doc]<br>-> Trình bày: 1. Luồng dữ liệu (Data flow từng bước) 2. Các hàm cốt lõi 3. Các điểm cần lưu ý khi phát triển tiếp.' |

## 3. 4 BÍ KÍP HIỆN ĐẠI NÂNG TRÌNH LÀM VIỆC VỚI AI (PRO TIPS)

| Kỹ thuật | Cách áp dụng hằng ngày | Hiệu quả mang lại |
|---|---|---|
| **1. Reverse Prompting (Hỏi ngược lại)** | Thêm vào cuối câu: 'Bạn có cần thêm thông tin gì để hoàn thành tốt nhất không?' | Tránh AI phỏng đoán sai bối cảnh hệ thống. |
| **2. Phân vai Chuyên gia (Persona Framing)** | Gán vai cụ thể: 'Senior Database Admin' hoặc 'Security Auditor'. | Câu trả lời sâu, chuẩn thực tế, không lý thuyết suông. |
| **3. Chia nhỏ (Divide & Conquer)** | Bước 1: Lên Outline / Interface -> Bước 2: Viết hàm -> Bước 3: Viết Test. | Kiểm soát 100% logic, không bị trôi ngữ cảnh. |
| **4. Tự phản biện (Self-Review)** | Sau khi AI sinh code, bảo: 'Hãy tìm 2 điểm yếu và 1 lỗ hổng trong code trên rồi vá lại.' | Loại bỏ 80% bug tiềm ẩn trước khi chạy thử. |